PRIZMATIC

Agent-Authoritative Project Management Service

Team Members:

Jeongmin Sohn(jeongmin.sohn@stonybrook.edu),

Heejoon Yi(heejoon.yi@stonybrook.edu)

Table of Contents

1. Introduction..... 3
1.1 Product Description..... 3
1.2 Scope..... 3
1.3 Users..... 4
1.4 User Feedback..... 5
1.5 Existing Alternatives..... 5
1.6 Definitions..... 6

2. Requirements..... 7
2.1 Functional Requirements..... 7
2.2 Use Cases..... 9
2.3 User Interfaces..... 12
2.4 Non-functional Requirements..... 12

3. System Architecture..... 13
3.1 Overview..... 13
3.2 Data Design..... 19
3.3 UML Sequence Diagrams..... 22
3.4 API Design..... 23
3.5 Deployment..... 23
3.6 Code Conventions..... 24

4. Schedule..... 26

5. Contributions..... 27

1. Introduction

1.1 Product Description

Prizmatic is a web-based project management platform designed for small software development teams, including a single developer. It provides an AI project manager to support efficient project management and aims to minimize inefficiencies in work distribution, progress tracking, and code collaboration processes that occur within a team.

Many small teams struggle with organizing work, tracking progress, and managing pull requests across multiple tools due to a lack of dedicated project managers. As a result, teams must manually track their project status and progress, often placing the burden on their most capable developers to act as de facto project managers. This significantly reduces their effectiveness in core development tasks, and the impact is even greater in smaller teams where the contribution of a single skilled developer is critical, ultimately leading to inefficient workflows and decreased productivity.

Prizmatic solves these problems by providing a centralized dashboard that offers a clear view of project progress, pull request activity, upcoming deadlines, and overall team workflow. In addition, the AI project manager analyzes the current status of the project to automatically assign tasks, recommend priorities, and track deadlines to keep everything on schedule. It also monitors pull requests and provides meaningful feedback.

By passing project management tasks to AI project managers, Prizmatic reduces the need for manual progress tracking and enables systematic project management, allowing developers to focus more on development work.

1.2 Scope

In Scope

This project focuses on developing a web-based AI-powered project management platform designed for small software development teams and individual developers.

The system includes:

- centralized project and task management
- collaboration features such as workspace and member management

- hierarchical work item organization and sprint-based workflow
- AI-assisted features such as PR review and backlog generation
- GitHub integration for development tracking

The system will be deployed as a web application and accessible via modern browsers (Chrome, Safari, and Windows Explorer) on desktop devices.

Out of Scope

The following features are excluded due to time constraints:

- Mobile application support
- Offline support
- Real-time collaborative editing

1.3 Users

The intended users of this system are small software development teams and individual developers. These users are expected to have varying levels of technical expertise but are generally familiar with basic software development workflows.

1. **Individual Developers**
   - Typically, students or independent developers working on personal or academic projects
   - Moderate technical background with familiarity in tools such as GitHub
   - May lack experience in project management and benefit from AI-assisted guidance
   - Prefer simple and intuitive interfaces with minimal setup

2. **Small Development Teams**
   - Groups of 2–10 developers
   - Members may have different roles and varying experience levels
   - Familiar with development workflows such as pull requests, task tracking, and sprint planning
   - Require coordination tools, but often lack a dedicated project manager

Usability Considerations

- The system should be easy to use for users with limited project management experience
- Clear UI design and guidance should be provided to reduce complexity

1.4 User Feedback

We can receive feedback at multiple stages of the development process to ensure that the system meets the needs of our intended users.

1. Design Stage

We can receive feedback through informal discussions with fellow students on initial UI mockups and core feature ideas, which helps validate whether the overall concept and interface are intuitive.

2. Development Stage

We can share a prototype with potential users and ask them to perform basic tasks, such as creating a workspace and managing tasks. Then, we may gather feedback through short surveys, mainly on usability, user experience, and workflow efficiency.

3. Beta Release Stage

We can also share a near-final version of our product with a small group of users and ask them to test it. After they have tested it for a significant amount of time, we may receive feedback focused on bugs, performance issues, and overall user experience.

1.5 Existing Alternatives

Several existing tools address project management for software development teams, including Jira and Notion. Each of these platforms provides useful features but also has limitations, especially in requiring manual project management rather than providing automated support.

1. Jira

- **Strengths:**
    - Highly customizable workflows and issue tracking
    - Support for agile methodologies (sprints, backlog management)
    - Widely used in industry
- **Weaknesses:**
    - Complex and overwhelming for small teams or beginners
    - Requires significant setup and configuration
    - Many features are unnecessary for simple projects

    - Lacks automation in project management decision-making

2. **Notion**
   - **Strengths:**
     - Flexible and easy-to-use interface
     - Supports note-taking, documentation, and basic task management
     - Highly customizable for different use cases
   - **Weaknesses:**
     - Lacks structured project management features, such as sprint tracking
     - Requires manual organization and maintenance
     - Limited automation for development workflows
     - Not designed specifically for software development workflows

1.6 Definitions

This section defines key terms and concepts used throughout this document.

Workspace: A shared environment where a group of users collaborates on projects.
Project: A collection of related tasks within a workspace.
WorkItem: A unit of work in the system, such as an epic, story, or task, used to organize and track progress.
Epic: A high-level feature or objective that represents a major functionality of the system.
Story: A smaller unit of work derived from an epic, typically representing a user requirement or feature.
Task: An actionable unit of work that can be completed within a short period of time.
Sprint: A fixed time period during which a set of work items is completed
Kanban Board: A visual board that displays work items in different stages (To Do, In Progress, In Review, Done).
Backlog: A prioritized list of work items that need to be completed in a project
Artifact: An external resource linked to the project.
Pull Request (PR): A request to merge code changes into a repository.
OAuth: An authentication method that allows users to log in using third-party accounts, such as Google and GitHub.

---

2. Requirements

2.1 Functional Requirements

Authentication & Account System

Users can create and manage accounts using email/password or OAuth (GitHub, Google).

- Users can sign up and log in
- The system ensures that each email address is unique
- The system allows users to securely authenticate
- Users can log out of the system
- Users can reset and change their passwords

User Profile

Users can view and update their profile information.

- Users can view their profile information
- Users can update their username and avatar
- Users can manage connected accounts (GitHub, Google)
- Users can configure notification preferences

Workspace Management

Users can participate in and access workspaces as the primary organizational unit.

- Users can create a workspace
- Workspace owners can invite members via email or link
- The system supports role-based access (Owner, Member, Viewer)
- Workspace owners can configure workspace settings

Project Management

Projects are created and managed within a workspace, serving as containers for organizing work, tasks, and team collaboration.

- Users can create projects
- Project owners can configure project settings
- The system supports assigning roles to users within a project (Owner, Member, Viewer)

WorkItem System

WorkItems are created and managed within a project, serving as units for organizing, tracking, and executing tasks.

- Users can create, update, and delete WorkItems
- Users can organize WorkItems in a hierarchical structure (Epic, Story, Task)
- Users can update the status of WorkItems (Todo, In Progress, In Review, Done)
- Users can comment on WorkItems and mention other users

Artifact System

Artifacts connect WorkItems with external resources.

- Users can attach URLs to WorkItems
- The system automatically classifies artifacts based on the URL

Sprint System

The system supports sprint-based workflow management.

- Users can create and manage sprints
- The system provides a Kanban-style sprint board
- Users can drag and drop tasks across statuses
- The system generates sprint reports upon completion

GitHub Integration

The system integrates with GitHub repositories.

- Users can connect repositories via GitHub App
- The system automatically links PRs to WorkItems
- The system parses WorkItem IDs from branch names or PR titles

PR Review Bot

The system provides AI-assisted code review.

- The system summarizes pull requests
- The system analyzes potential risks
- The system suggests improvements and tests
- The system categorizes feedback as suggestion, concern, or blocker

AI Project Manager

The system provides intelligent project management assistance.

- The system generates WorkItems
- The system detects duplicate tasks
- The system suggests task decomposition
- The system recommends task assignments
- The system assists with sprint planning
- The system generates daily summaries

Dashboard & Notifications

- The system provides a dashboard with project insights
- The system notifies users of relevant events through multiple channels (in-app notifications and email).

2.2 Use Cases

Use Case 1: Create Workspace and Invite Members

- Primary Actor: Workspace Owner
- Priority: Essential
- Goal in Context: The user wants to create a new workspace and begin collaborating with other team members.
- Preconditions: The user is logged in.
- Scenario:
    1. The user selects the option to create a new workspace.
    2. The system prompts the user to enter the workspace information, such as name, and description.
    3. The user submits the workspace information form.
    4. The system assigns the user as the workspace owner and creates the workspace.
    5. The owner selects the option to invite members.
    6. The system prompts the user to invite members via email or invite link.
    7. The user enters member email addresses or copies the invite link.
    8. The system sends invitations or generates the invite link
    9. The invited users join the workspace through the invitation link.
- Extensions:

  3a. If the workspace name is invalid or empty, the system displays an error message and asks the user to correct it.

4a. If the workspace creation fails, the system notifies the user and allows the user to retry.

7a. If the email format is invalid, the system displays an error message and asks the user to correct it.

8a. If sending the invitation fails, the system notifies the user and allows the user to retry.

Use Case 2: Create and Organize WorkItems

- Primary Actor: Project Owner/Member
- Priority: Essential
- Goal in Context: The user wants to create and organize project work into a structured hierarchy.
- Preconditions: A project already exists and the user is a member of the project.
- Scenario:
    1. The user opens a project
    2. The user selects the option to create a new WorkItem.
    3. The system prompts the user to enter details such as title, description, priority, assignee, labels, status, and hierarchy level (Epic, Story, or Task).
    4. The user submits the WorkItem information.
    5. The system creates the WorkItem.
    6. The system displays the WorkItem in the project backlog with the assigned hierarchy.
- Extensions:

  3a. If required fields are missing, the system displays an error message and asks the user to correct it.

  5a. If the WorkItem creation fails, the system notifies the user and allows the user to retry.

Use Case 3: Plan and Manage a Sprint

- Primary Actor: Project Owner
- Priority: Essential
- Goal in Context: The user wants to organize selected WorkItems into a sprint and track progress on the board.
- Preconditions: The user has project owner permissions and the project contains WorkItems in the backlog.

- Scenario:
    1. The user selects the option to create a sprint.
    2. The system prompts the user to enter sprint details, such as name, start date, and end date.
    3. The user enters and submits the sprint information.
    4. The system creates the sprint.
    5. The user selects WorkItems from the backlog and adds them to the sprint.
    6. The system displays the sprint board under designated columns according to its current status (To Do, In Progress, In Review, and Done).
    7. Team members update WorkItem statuses as work progresses.
    8. The system updates the sprint board in response to status changes.
- Extensions:

  3a. If the sprint dates are invalid, the system displays an error message and asks the user to correct them.

  4a. If sprint creation fails, the system notifies the user and allows the user to retry.

  7a. If a user without permission attempts to update a WorkItem status, the system prevents the action.

Use Case 4: Link Pull Requests to WorkItems

- Primary Actor: Project Member
- Secondary Actor: GitHub
- Priority: Expected
- Goal in Context: The user wants development activity in GitHub to be automatically linked to WorkItems.
- Preconditions: The user has access to the project and the project is connected to a GitHub repository by the owner.
- Scenario:
    1. The user creates a branch or pull request in GitHub that includes a WorkItem ID in its name or title.
    2. GitHub sends a pull request event to the system.
    3. The system receives the event and extracts the WorkItem ID.
    4. The system links the pull request to the corresponding WorkItem.
    5. The system displays the linked pull request in the WorkItem details.
- Extensions:

3a. If no WorkItem ID is found, the system does not create a link.

4a. If the referenced WorkItem does not exist, the system notifies the user to resolve it manually.

Use Case 5: View AI Insights and Reflect on them

- Primary Actor: Project Owner / Member
- Priority: Nice to Have
- Goal in Context: The user wants to review AI-generated insights to understand project status and make better decisions.
- Preconditions: The user has access to the project and the system has already generated insights based on project data
- Scenario:
    1. The user opens the project dashboard.
    2. The system displays AI-generated insights, including task priorities, potential delays, and workload distribution.
    3. The user reviews the insights.
    4. The user takes actions based on the insights, such as reprioritizing tasks or reassigning WorkItems.
    5. The system updates the project accordingly.
- Extensions:

  2a. If the insights cannot be generated due to lack of the project information, the system displays a message indicating that insights are not yet generated.

  2b. If the insights cannot be loaded, the system displays an error message and allows the user to retry.

2.3 User Interfaces

https://v0-prism-feature-specification.vercel.app/

2.4 Non-functional Requirements

Performance: The system should respond within 2 seconds

Security: All data must be transmitted over HTTPS

Reliability: The system should handle invalid inputs and error cases

Usability: The system should provide clear error messages and the UI should be simple so that beginners can also easily use
Compatibility: The system should support modern browsers, such as Chrome, Safari, and Windows Explorer

---

3. System Architecture

3.1 Overview

PRISM is a web-based project and workspace management system. The system is divided into a frontend web application, a backend REST API server, a PostgreSQL database, and optional cloud infrastructure components for email, queueing, and object storage. The main user-facing features are authentication, onboarding, workspace management, project management, project member management, and work item tracking.

The frontend is implemented as a Next.js application that renders the user interface, manages user interaction, stores short-lived authentication state, and communicates with the API server. The backend is implemented as a NestJS application that handles business rules, authentication, authorization, validation, transactional workflows, and database access. PostgreSQL stores relational project data such as users, authentication records, workspaces, workspace members, projects, project members, project jobs, work items, labels, invitation records, and refresh tokens.

Major Components

Component	Languages and Technologies	Responsibility
Frontend	TypeScript 5.9.3, Next.js 16.1.1, React 19.2.3, Tailwind CSS 4.1.18	User interface, routing, client-side interaction, auth session handling, API calls
Backend	TypeScript 5.7.3, NestJS 11, Node.js 22 Docker runtime	REST API, authentication, authorization, validation, business workflows, integrations

Database	PostgreSQL 16 planned target, TypeORM 0.3.20, pg 8.16.3	Relational persistence, transactional data, users, workspaces, projects, work items
Cloud services	Oracle Cloud Infrastructure SDK 2.129.0	Optional email delivery, object storage, and queue processing
Tooling	pnpm 10.26.1, ESLint 9, Prettier 3, Husky	Dependency management, linting, formatting, CI checks, git hooks

Third-Party Components and Intended Usage

Component	Version	Used In	Intended Usage
Node.js	24.12.0 frontend local target, 22 Alpine backend Docker runtime	Full stack runtime	JavaScript and TypeScript runtime environment
pnpm	10.26.1	Package management	Dependency installation and script execution
Next.js	16.1.1	Frontend	App Router, page routing, server/client rendering, API route support
React	19.2.3	Frontend	Component-based UI implementation
TypeScript	5.9.3 frontend, 5.7.3 backend	Full stack	Static typing and maintainable application contracts
Tailwind CSS	4.1.18	Frontend	Utility-first styling system
Radix UI	1.x and 2.x packages	Frontend	Accessible base UI components such as dialogs, dropdowns, checkboxes, labels, and tooltips
TanStack Query	5.90.12	Frontend	API request state, caching, mutations, and loading/error handling
Axios	1.13.2	Frontend	HTTP client for communicating with the NestJS API

Zustand	5.0.9	Frontend	Lightweight local UI state management
Zod	4.3.6	Frontend	Runtime validation for environment variables and structured data
NestJS	11.x	Backend	Modular REST API framework with dependency injection
TypeORM	0.3.20	Backend and database	PostgreSQL connection, transactions, migrations, and query execution
PostgreSQL	16 planned target	Database	Relational storage for users, workspaces, projects, work items, memberships, labels, and tokens
PostgreSQL Driver pg	8.16.3	Backend and database	Node.js PostgreSQL connectivity
Argon2	0.44.0	Backend	Secure password hashing
@nestjs/jwt	11.0.2	Backend	JWT access and refresh token support
Helmet	8.1.0	Backend	Security-related HTTP headers
@nestjs/throttler	6.5.0	Backend	API rate limiting
Swagger	11.2.0 Nest package, 5.0.1 UI package	Backend	API documentation
Google Auth Library	10.6.2	Backend	Google OAuth token verification
Octokit / GitHub OAuth	Octokit 5.0.5, auth-oauth-user 6.0.2	Backend	GitHub OAuth authentication support
OCI SDK	2.129.0	Backend infrastructure	Email delivery, queue, and object storage integrations

Design Patterns and Architectural Style

The overall system follows a modular full-stack architecture. The frontend uses a domain-oriented structure and Atomic Design for UI organization. This makes feature code

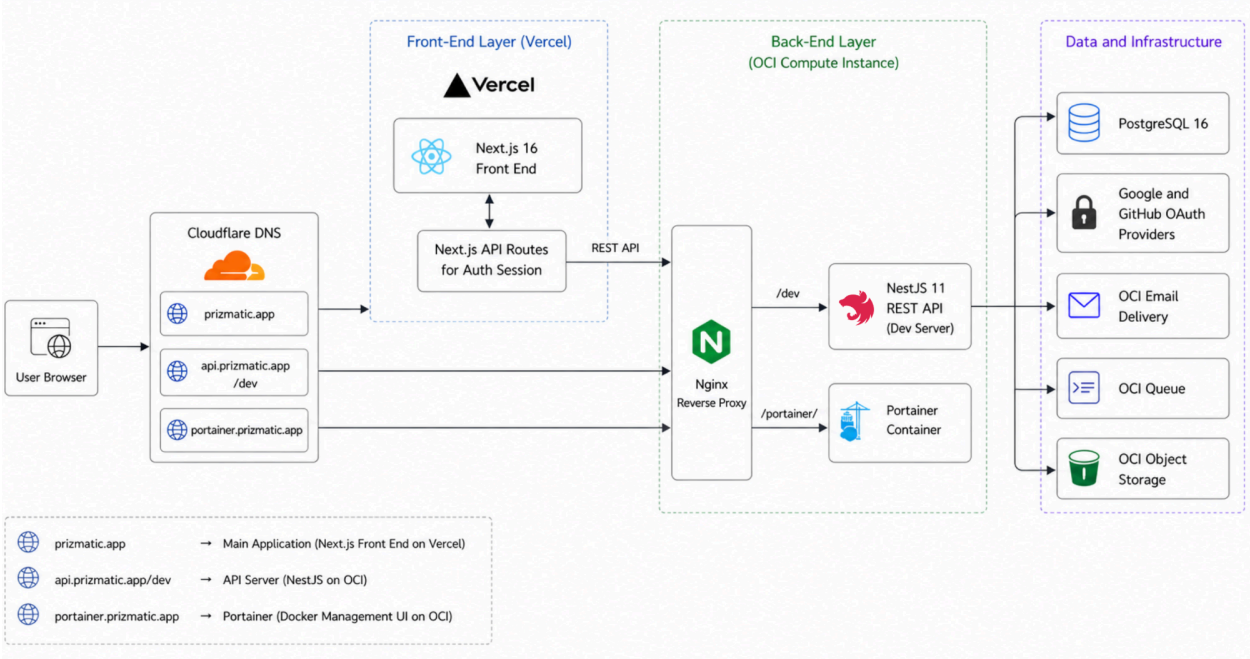
easier to locate while keeping reusable visual components separate from business-specific screens.

It also utilizes a prefetching method, to fully enable the power of server-side-rendering (SSR). Separating server components and client components makes it more resilient in terms of User Experience (UX), as the loading time is much shorter compared to typical client-side-rendering (CSR).

The backend follows a layered clean architecture rather than a strict Model-View-Controller pattern. The main flow is controller to use case to service or repository to database. Controllers handle HTTP transport concerns, use cases coordinate application workflows, services contain reusable business or infrastructure logic, and repositories encapsulate persistence logic.

The project also uses the Repository pattern to isolate database access, Dependency Injection through NestJS providers, DTOs for API request and response contracts, Guards and Interceptors for cross-cutting API behavior, and a Unit of Work pattern for transactional operations.

High-Level Architecture Diagram



The system is designed with a clear separation between DNS-level routing and application-level routing. At the edge, Cloudflare DNS handles routing strictly at the subdomain level:

- prizmatic.app → frontend hosted on Vercel
- api.prizmatic.app → backend running on OCI
- portainer.prizmatic.app → Portainer management UI

No path-based routing is handled at the DNS layer.
The frontend is deployed on Vercel using Next.js, with authentication partially managed via Next.js API routes, and communicates with the backend through REST APIs.
On the backend, services run on an OCI Compute instance, where Nginx acts as a reverse proxy. Path-based routing is handled internally:

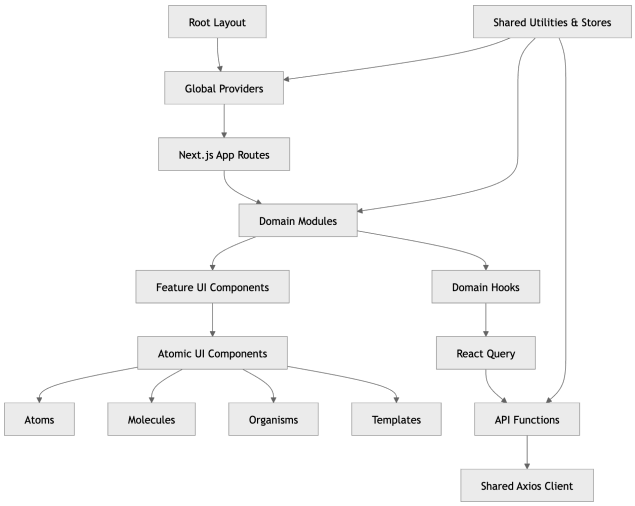
- api.prizmatic.app/dev is forwarded to the NestJS REST API server

Portainer is exposed directly via its dedicated subdomain (portainer.prizmatic.app), not through a path.
The backend is fully containerized, and integrates with managed services including PostgreSQL, OAuth providers (Google, GitHub), OCI Email Delivery, OCI Queue, and OCI Object Storage.
Overall, the architecture separates:

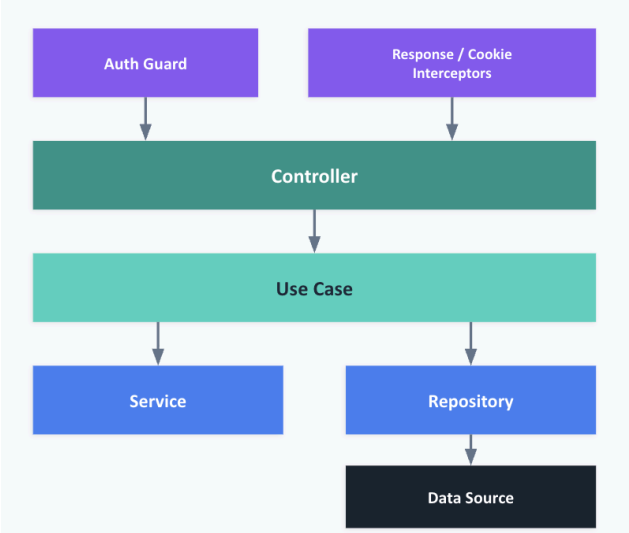
- Edge routing (Cloudflare)
- Frontend delivery (Vercel)
- Backend routing and services (OCI + Nginx + containers)
- Managed infrastructure and data services

This structure enables clean routing boundaries, operational flexibility, and scalability.

Frontend Architecture



Backend Architecture



Technology Stack Rationale

This technology stack was chosen because it provides a consistent TypeScript-based development experience across both the client and server. Using TypeScript on both sides reduces context switching and makes it easier to maintain shared concepts such as users, workspaces, projects, work items, and API payloads.

Next.js is appropriate for the frontend because it supports modern React development, file-based routing, server-side capabilities, and production-ready optimization. NestJS is appropriate for the backend because its module system, dependency injection, guards, interceptors, and validation tools provide a clear structure for a growing API.

PostgreSQL was selected because the system data is relational and requires reliable joins, constraints, transactions, and query performance. TypeORM gives the application a standard integration layer for PostgreSQL while still allowing direct SQL when complex queries are easier to express explicitly.

The selected third-party libraries reduce implementation risk. Radix UI improves accessibility, React Query simplifies remote state management, Argon2 strengthens password security, Helmet and rate limiting improve the API security baseline, and Swagger makes API integration easier. OCI services are included as optional managed infrastructure so the system can support email, queueing, and object storage without building those services from scratch.

3.2 Data Design

The original Data Design artifact is stored at prism-nestjs/Documents/Prizmatic-DataDesign.pdf. The design below updates that material to match the current implementation in the NestJS repositories and DTOs. The database uses PostgreSQL with UUID primary keys and snake_case table and column names. The application layer maps database fields to camelCase DTO fields before returning data to the frontend.

Entity Summary

Table	Purpose
prism_users_l	Stores user profiles, including email, full name, username, timestamps, and soft-delete metadata
prism_user_auths_l	Stores authentication provider records for email, Google, and GitHub sign-in
prism_email_tokens_l	Stores email verification tokens and usage status
prism_refresh_tokens_l	Stores hashed refresh tokens, expiration timestamps, and revocation timestamps
prism_workspaces_l	Stores workspace metadata, owner, slug, description, timezone, locale, status, and soft-delete state
prism_workspace_members_l	Connects users to workspaces with roles such as admin, member, and viewer
prism_workspace_invitations_l	Stores invitation tokens for adding users to workspaces
prism_workspace_invitation_events_l	Stores invitation lifecycle events such as accepted or declined
prism_projects_l	Stores projects under workspaces, including slug, description, timezone, and locale
prism_project_jobs_l	Stores workspace-level job roles that can be assigned to project members
prism_project_members_l	Connects users to projects
prism_project_member_job_map	Many-to-many mapping between project members and project jobs
prism_work_items_l	Stores project work items such as epics, stories, and tasks

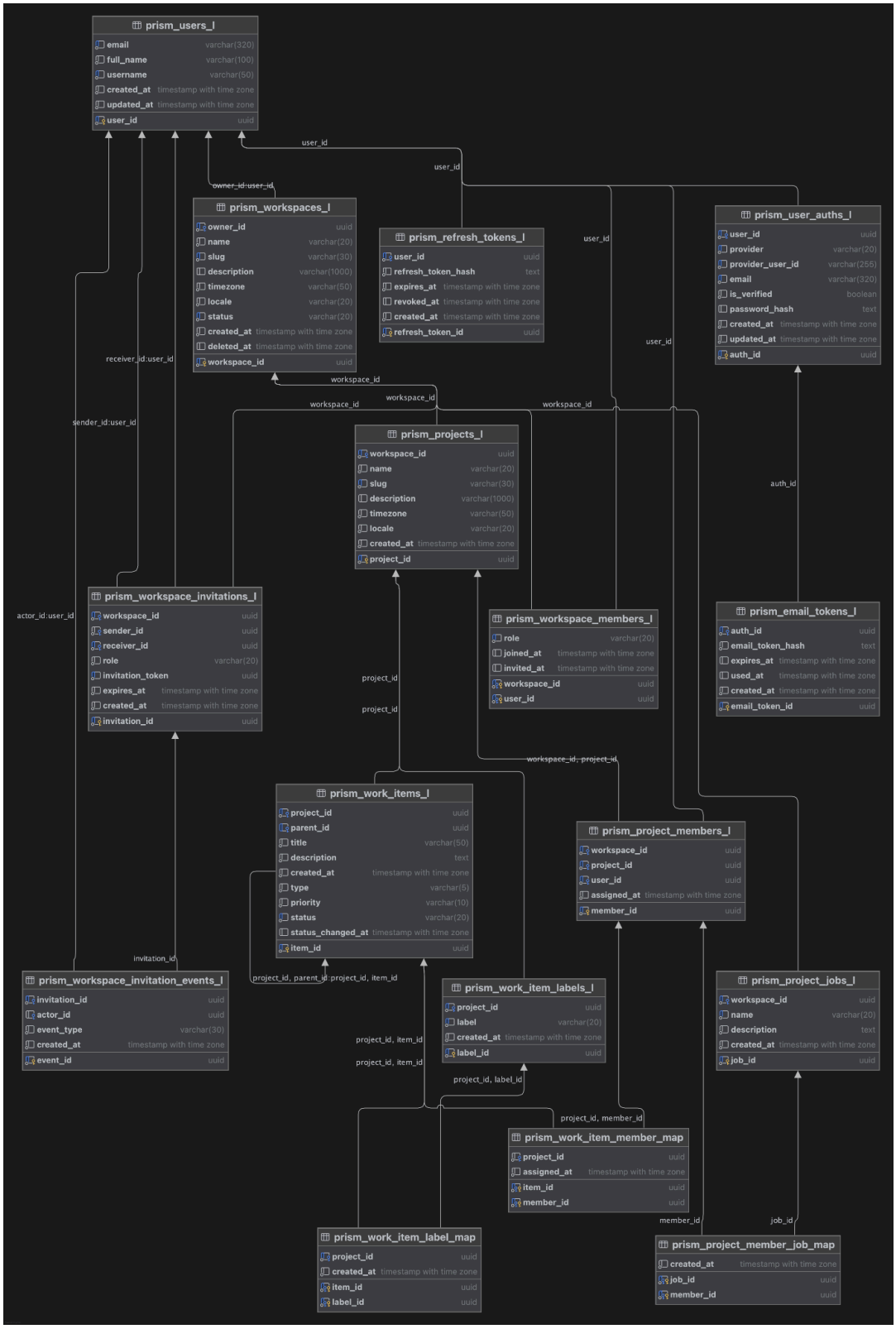
prism_work_item_member_map	Many-to-many mapping between work items and assigned project members
prism_work_item_labels_l	Stores reusable labels within a project
prism_work_item_label_map	Many-to-many mapping between work items and labels

Main Relationships

Users can have multiple authentication records, refresh tokens, workspaces, workspace memberships, and project memberships. A workspace belongs to one owner but can have many members. A workspace contains many projects and project job definitions. A project contains many project members and work items. A work item belongs to one project and can optionally reference another work item as its parent. Work items can have multiple assignees and multiple labels through mapping tables.

Workspace invitations connect a sender, receiver, and workspace through an invitation token. Invitation events record whether the invitation has been accepted, declined, or otherwise acted upon. Refresh tokens and email tokens are stored separately from user profiles so authentication state can be revoked or expired without modifying the core user record.

Data Design ERD



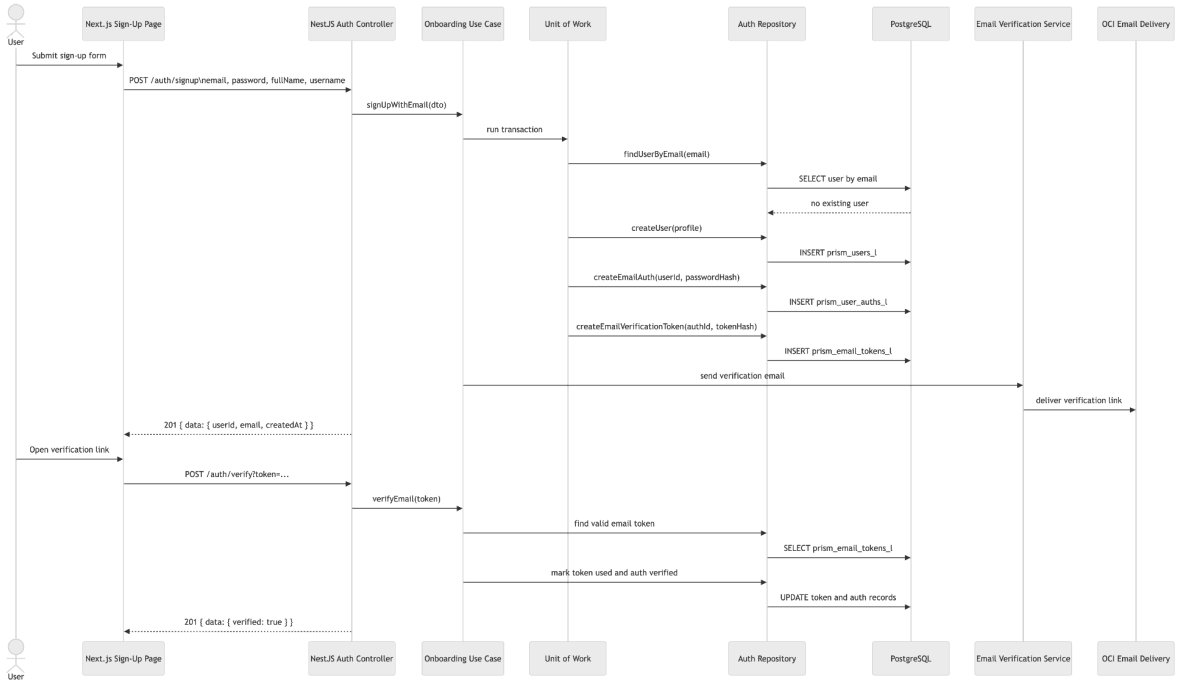
Data Integrity Notes

The system uses UUID identifiers for primary keys and foreign keys. Workspace roles are restricted to admin, member, and viewer. Work item types are restricted to epic, story, and task. Work item priorities are restricted to low, medium, high, and urgent. Work item statuses are restricted to todo, in_progress, in_review, and done.

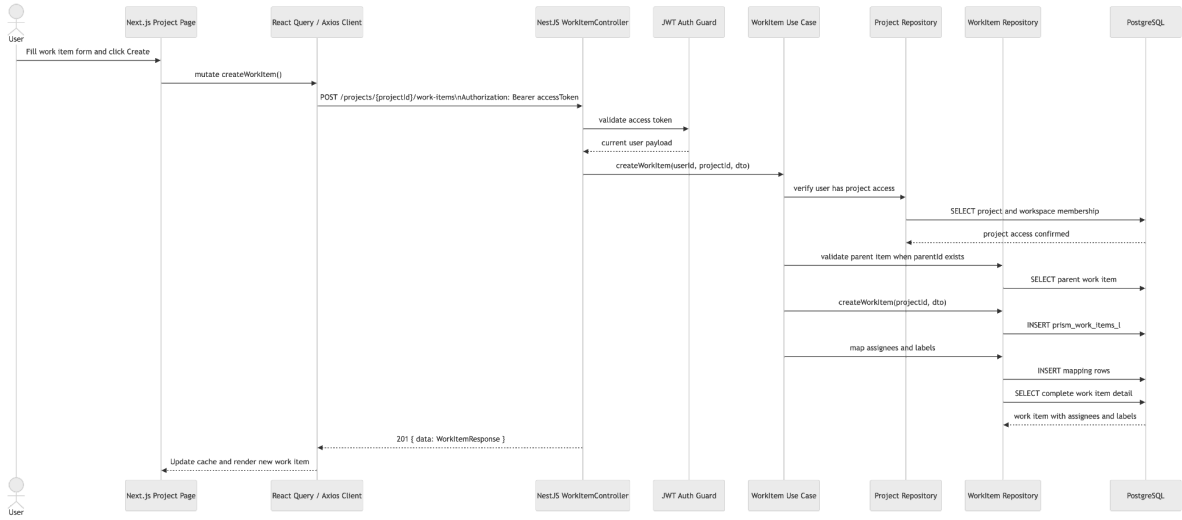
The backend disables automatic TypeORM schema synchronization and plans to manage schema changes through migrations. Application workflows that modify multiple tables use the Unit of Work pattern so related database changes are committed or rolled back together.

3.3 UML Sequence Diagrams

Use Case 1: Email Sign-Up and Verification



Use Case 2: Create Work Item in a Project



3.4 API Design

https://docs.google.com/spreadsheets/d/17DLFtYUFssA0XaqfG2R3DuctJp2EOpavnpMULekUpkU/edit?usp=sharing

3.5 Deployment

The frontend will be deployed as a Next.js application on Vercel. Vercel is a good fit because it provides built-in support for Next.js routing, server-side rendering, API routes, environment variables, preview deployments, and HTTPS. The frontend will use environment variables such as NEXT_PUBLIC_API_HOST and NEXT_PUBLIC_SITE_URL to connect to the deployed API server.

The backend will be deployed as a Dockerized NestJS service. The repository already includes a multi-stage Dockerfile using Node.js 22 Alpine, so the deployment target can run the built server with node dist/src/main.js. The planned deployment target is an OCI Compute instance or OCI container-based service behind HTTPS. Required environment variables include database credentials, JWT secrets, CORS origins, OAuth client settings, and optional OCI integration settings.

The database will be deployed as a managed PostgreSQL 16-compatible database. A managed database is preferred over hosting PostgreSQL manually because it simplifies backups, monitoring, upgrades, and recovery. The NestJS server will connect to it through TypeORM using PG_HOST, PG_PORT, PG_USER, PG_PASSWORD, PG_DB, and optional PG_SCHEMA.

Optional infrastructure services will use Oracle Cloud Infrastructure because the backend already includes OCI SDK integrations. OCI Email Delivery will be used for email verification

and workspace invitation emails. OCI Object Storage will be used if the project needs uploaded files or generated assets. OCI Queue will be used if the team adds asynchronous background processing.

GitHub Actions will be used for continuous integration. The frontend checks should run pnpm lint, pnpm type-check, and pnpm build. The backend checks should run pnpm run lint:check, pnpm run test, and pnpm run build. Deployment secrets will be stored in the deployment platform rather than committed to the repository.

3.6 Code Conventions

The project will follow TypeScript conventions enforced by ESLint and Prettier. The frontend uses Next.js ESLint rules and TypeScript checks, while the backend uses typescript-eslint, recommended ESLint rules, and Prettier. The team will also use the Airbnb JavaScript Style Guide as the general JavaScript/TypeScript style reference: https://github.com/airbnb/javascript. Additional tooling references are TypeScript ESLint at https://typescript-eslint.io/ and Prettier at https://prettier.io/docs/.

Naming Conventions

Item	Convention	Example
TypeScript variables and functions	camelCase	createWorkspace, accessToken
React components	PascalCase	Sidebar, MainLayout
Classes and DTOs	PascalCase with descriptive suffixes	CreateWorkspaceDto, ProjectRepository, AuthUseCase
Hooks	use prefix with camelCase	useApiQuery, useCurrentUser
Constants	UPPER_SNAKE_CASE	WORKSPACE_MEMBER_ROLES
Database tables and columns	snake_case	prism_workspaces_l, workspace_id
API URLs	plural RESTful resource names	/workspaces/:workspaceId/members

File names	existing project style by layer	workspace.controller.ts, work-item.usecase.ts, Button.tsx

Formatting Conventions
Frontend formatting follows prism-nextjs/.prettierrc.json: 2 spaces, no tabs, 120-character print width, double quotes, trailing commas, and no parentheses around single-argument arrow functions when possible. Backend formatting follows prism-nestjs/.prettierrc: single quotes, trailing commas, and LF line endings.

Code should be organized by responsibility. Frontend feature code should stay under src/domains, reusable UI should stay under src/atomics, and cross-feature utilities should stay under src/shared. Backend routes should stay in controllers, workflow logic in use cases, reusable domain or infrastructure behavior in services, database access in repositories, and request/response contracts in DTOs.

Comments should be used only when they clarify non-obvious business rules, security behavior, or complex queries. The team should avoid comments that simply repeat what the code already says. API and database changes should be reflected in DTOs, Swagger annotations, and this API design section.

Style Checking
The frontend should be checked with:
- pnpm lint
- pnpm type-check
- pnpm build

The backend should be checked with:
- pnpm run lint:check
- pnpm run test
- pnpm run build

Layering Conventions

The system follows a strict layering convention to enforce separation of concerns, scalability, and long-term maintainability across both frontend and backend.

At a high level, the architecture is divided into three major areas: domains, shared, and presentation layers.

The core business logic lives inside domain boundaries. Each domain is treated as an isolated unit that owns its own components, API logic, types, and internal workflows. Cross-domain dependencies are intentionally restricted. A domain must not directly depend on another domain. If logic needs to be reused, it is extracted into a shared layer instead. This prevents hidden coupling and keeps the system composable as it grows.

The shared layer acts as a foundation. It contains domain-agnostic utilities such as query abstractions, hooks, constants, and low-level helpers. This layer is strictly dependency-safe, meaning it does not depend on any domain or UI layer. It exists purely to support reuse without introducing circular dependencies.

On the presentation side, the UI is structured using an Atomic Design approach. Components are organized by composition depth rather than feature ownership. Lower-level primitives like atoms are kept completely generic, while higher-level components progressively compose them into more complex structures. Importantly, components always move upward in the hierarchy based on their dependencies, never sideways. This prevents ambiguous ownership and enforces a predictable UI structure.

Any UI component that contains domain-specific knowledge is explicitly placed inside its corresponding domain instead of the shared UI layer. This ensures that business context does not leak into generic presentation code.

Data flow follows a server-first model. Pages are treated as thin entry points, delegating data fetching to server components and passing the result down into a single client boundary. The client layer consumes this data through a standardized query abstraction, ensuring consistency in caching and state management. This approach removes the need for complex hydration strategies and keeps data flow explicit and predictable.

On the backend, the same principle of strict layering applies. Each feature module encapsulates its own controllers, use cases, services, and repositories. Controllers remain thin and handle only HTTP concerns. Use cases orchestrate business logic and define transaction boundaries. Repositories own persistence and SQL. Services handle side effects and external integrations. This layered separation ensures that business logic is not fragmented across the system and remains testable and maintainable.

Overall, this layering convention enforces a few critical invariants: business logic is isolated, dependencies flow in one direction, and shared abstractions remain truly reusable.

---

4. Schedule

Project Milestone 1 (5/11/2026) focuses on establishing the core product foundation. At this stage, the goal is to deliver a functional application where users can navigate projects and interact with tasks. This includes implementing the project detail page, task detail page, and completing the basic WorkItem CRUD flow. Heejoon Yi is responsible for developing the Project API and the corresponding frontend pages, as well as the task management UI. In parallel, Jeongmin Sohn focuses on implementing the remaining backend components, including Task

APIs and related endpoints. The frontend and backend integration is stabilized to support a complete, end-to-end user flow without relying on placeholder logic.

Project Milestone 2 (5/13/2026) introduces collaboration features and real-time capabilities. Sprint functionality is implemented, including a board-based workflow for managing tasks. A WebSocket layer is integrated to enable real-time updates across clients. Responsibilities are split across layers: Jeongmin Sohn handles the server-side WebSocket implementation, while Heejoon Yi implements the client-side integration and UI updates. Task and sprint state changes are reflected in real time, forming the foundation for collaborative interaction.

Project Milestone 3 (5/20/2026) focuses on activating the agent system and its supporting data pipeline. Heejoon Yi leads the development of the agent system, including the design and implementation of the event-driven agent loop. This also includes building the vector processing worker responsible for document chunking and embedding, as well as integrating with the vector database. A scheduler-based mechanism is introduced to trigger agent workflows. Meanwhile, Jeongmin Sohn continues to support backend stability and API extensions required to integrate agent-driven operations. The objective at this stage is to ensure that the agent system is fully operational, even if its capabilities are still evolving.

Project Milestone 4 (5/27/2026) completes the product by delivering its key differentiating features. Heejoon Yi implements the GitHub PR review bot and leads the development of the real-time human-to-agent chat system, integrating the agent into actual collaboration workflows. Jeongmin Sohn supports the backend infrastructure required for these features, including APIs for comments, messaging, and system interactions.

In addition, production deployment is handled by Heejoon Yi through an automated CI/CD pipeline built with GitHub Actions. Any merge into the **main** branch triggers the deployment workflow. For the backend, this includes building the Docker image, pushing it to GitHub Container Registry, pulling the latest image on the OCI Compute instance, restarting services via Docker Compose, and performing health checks. The frontend is automatically deployed through Vercel’s Git-based integration.

Final polishing and system integration are performed collaboratively to ensure a cohesive and production-ready user experience.

---

5. Contributions

Jeongmin Sohn:

- Implemented the Workspace backend API, including workspace creation, update, deletion, and member management logic.
- Developed the Workspace frontend pages, including UI for workspace creation, settings, and member invitations.
- Frontend-Backend integration and debugging, ensuring smooth data flow across the system
- Wrote and organized documentation.

Heejoon Yi:

- Project API Implementation & Integration
- Frontend Architectural Refactoring
- External Modules Implementation (Queue, Object Storage)
- AGENTS.md revisions to enforce AI agents like Codex to follow conventions strictly.
- Persistent User Session Implementation via proxy & JWT refresh token.